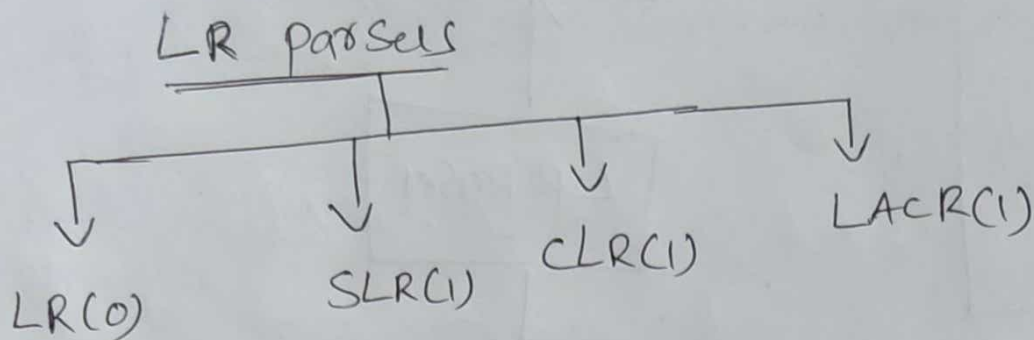


UNIT-5



→ LR(0) parser -

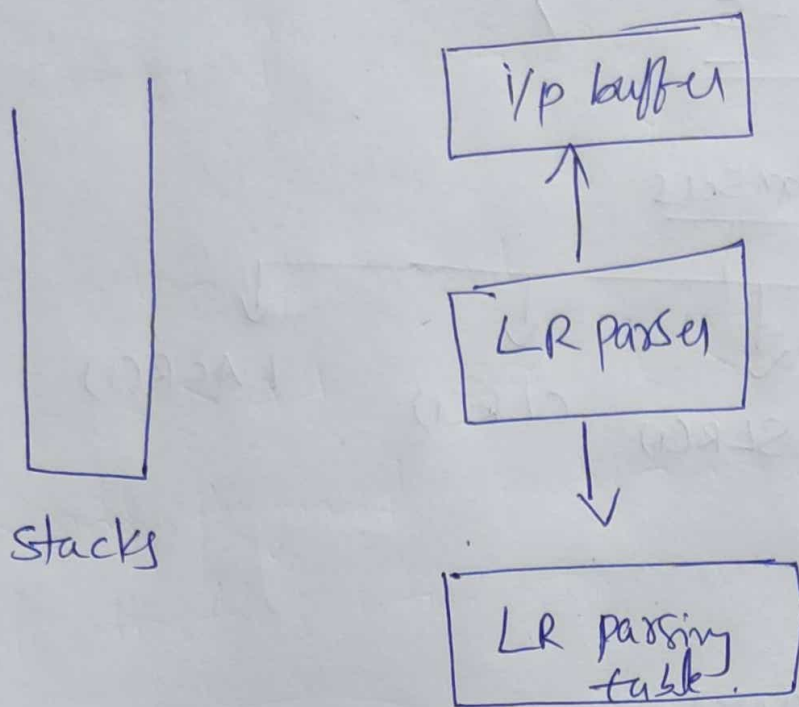
→ In the LR parsing 'L' stands for Left to right scanning of the input.

→ 'R' stand for Constructing a right most derivation in reverse.

→ "K" is the number of input symbol of the look ahead used to make number of parsing decision.

→ LR(0) algorithm - The LR algorithm requires Stack, input, output and parsing table.

→ In all types of LR parsing, input, output and Stack are same but parsing table is different.



- I/p buffer is used to indicate end of the input and it contains the string to be passed followed by a \$ symbol.
- A stack is used to contain a sequence of grammar symbols with a \$ at the bottom of the stack.
- passing table is two dimensional array. It contains two parts: Action part and GOTO part.

Various steps involved in the LR(0) parsing

- Check the ambiguity of the Grammar.
- Add Augment production in the given grammar
- Create canonical collection of LR(0) items
- Draw the Data flow diagram (DFA)
- Construct LR(0) Parsing table.

Augment Grammar - Augmented grammar G' will be generated if we add one more production in the given grammar G . It helps the parser to identify when to stop the parsing and announce the acceptance of the input.

Ex:-

$$S \rightarrow AA$$

$$A \rightarrow aA/b$$

$$S' \rightarrow S$$

$$S \rightarrow AA$$

$$A \rightarrow aA/b$$

→ Canonical collection of LR(0) items:

→ Given grammar

$S \rightarrow AA$, ~~and~~ $A \rightarrow aA/b$

→ add Augment production and insert '.' symbol at first position for every production in G.

$S' \rightarrow \cdot S, \$$
 $S \rightarrow \cdot AA$
 $A \rightarrow \cdot aA$
 $A \rightarrow \cdot b$

→ It is I_0 state.

→ $I_0 = \text{closure}(S' \rightarrow \cdot S)$

Add all production starting with S in I_0 to I_0 state b/c '.' is followed by the non-terminal.

→ $I_1 = \text{Goto}(I_0, S) = \text{closure}(S' \rightarrow S \cdot \$)$
 $= S' \rightarrow S \cdot \$$

$I_2 = \text{Goto}(I_1, \$) = \text{closure}(S' \rightarrow S \$ \cdot)$
 $= S' \rightarrow S \$ \cdot$

$I_3 = \text{Goto}(I_0, A) = \text{closure}(S \rightarrow A \cdot A)$. add all production starting with A into I_3 state. b/c '.' is followed by non-terminal.

Drawing DFA

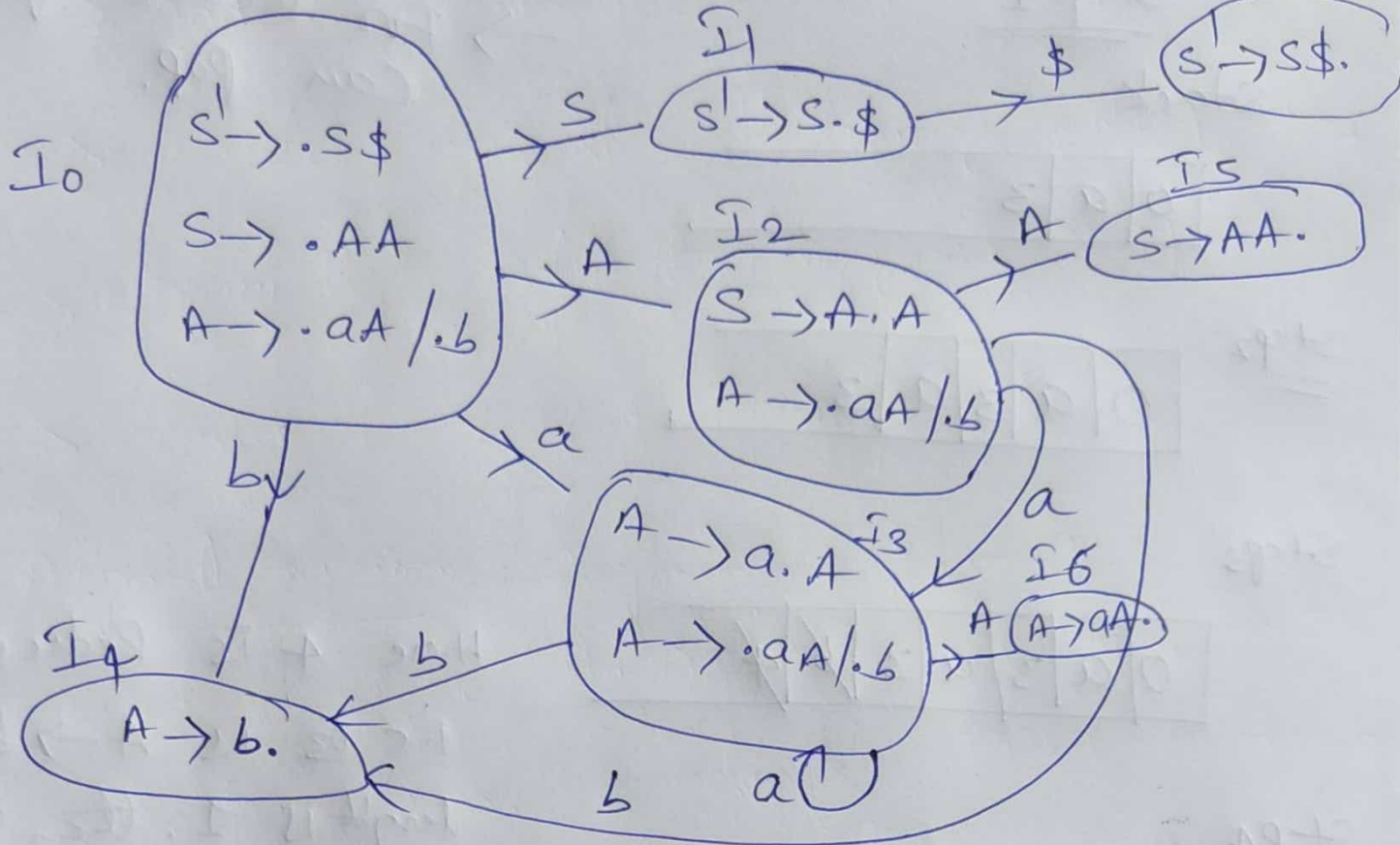
for $S' \rightarrow S$

$S \rightarrow AA$ - (1)

$A \rightarrow aA$ - (2)

$A \rightarrow b$ - (3)

accept



parsing table:

	Action			Goto	
	a	b	\$	A	S
0	S3	S4		2	1
1			accept		
2	S3	S4		5	
3	S3	S4		6	
4	δ_3 $A \rightarrow b$	δ_3 $A \rightarrow b$	δ_3 $A \rightarrow b$		
5	δ_1 $S \rightarrow AA$	δ_1 $S \rightarrow AA$	δ_1 $S \rightarrow AA$		
6	δ_2 $S \rightarrow aA$	δ_2 $S \rightarrow aA$	δ_2 $S \rightarrow aA$		

→ If input is $aabb\$$. is it accept (s) or not.

$aabbb\$$
 $\uparrow \uparrow \uparrow$

→ For shift you can push

→ For Reduce you can pop.

Step 1:

Stack

0	a	3	
---	---	---	--

Step 2:

0	a	3	a	3	
---	---	---	---	---	--

Step 3

0	a	3	a	3	b	4
---	---	---	---	---	--------------	--------------

Here 4 is Reduce

i.e 3 is $A \rightarrow b$

length is 1. then you

can pop 2 symbols from stack and

Step 4

0	a	3	a	3	A	6
---	---	---	---	---	---	---

→ Here '6' is Reduce. Push 'A' symbol

$A \rightarrow aA$ - length - 2 Pop (4)

Step 5

0	a	3	a	3	A	6
---	---	---	--------------	--------------	--------------	--------------

Step 6:

0	a	3	A
---	---	---	---

Step 7:

0	a	3	A	6
---	---	---	---	---

you can pop 4 items from stack.

Step 8:-

0	A	3	A	6
---	--------------	--------------	--------------	--------------

Step 9:-

0	A
---	---

a a b b \$
↑ ↑ ↑ ↑

Step 10:-

0	A	2	b	4
---	---	---	---	---

4 is reduction

Step 11:-

0	A	2	b	4
---	---	---	--------------	--------------

A → b

Step 12

0	A	2	A
---	---	---	---

Step 13

0	A	2	A	\$
---	--------------	--------------	--------------	---------------

S is reduce

S → AA, pop(4)

Step 14

0	S	1	\$
---	---	---	----

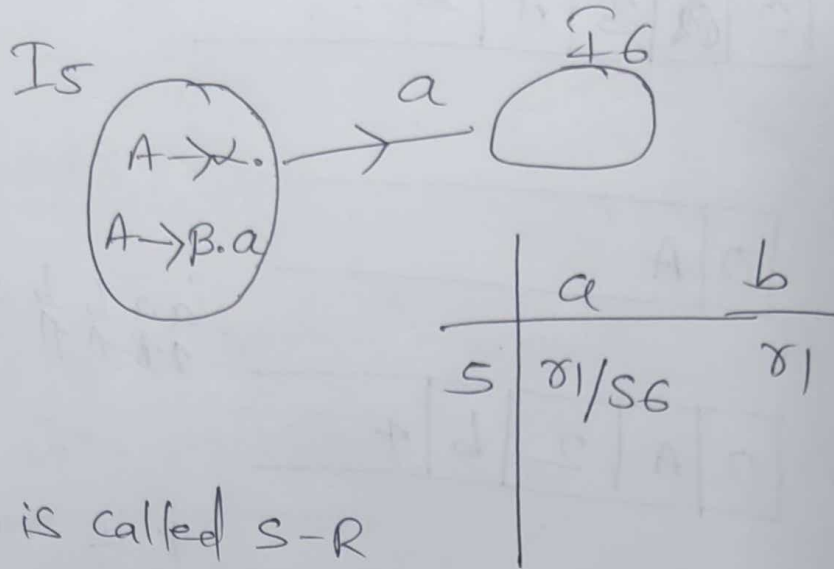
Step 15

0	S	accept
---	---	--------

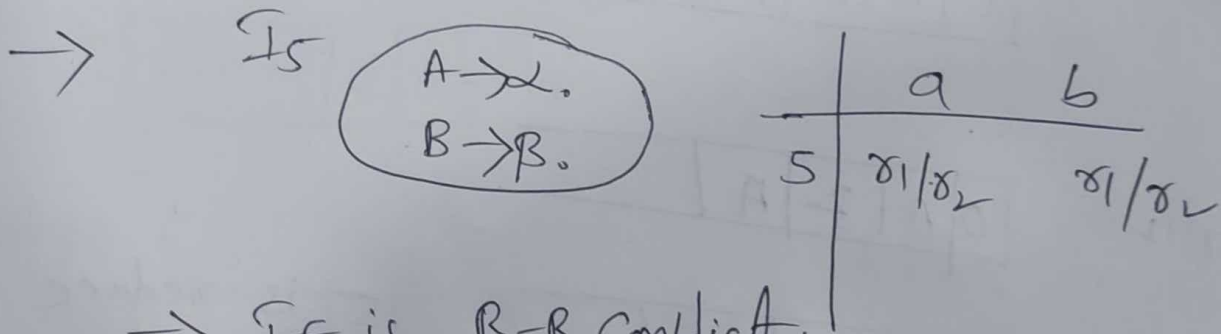
→ so the given string is accepted!

Conflicts in LR(0) parsing

eg:-



→ I_5 is called S-R conflict.



→ I_5 is R-R conflict.

SLR(1) Parsing -

- SLR(1) refers to simple LR parsing. It is same as LR(0) parsing.
- The only difference is in the parsing table.
- To construct SLR(1) parsing table, we use canonical collection of LR(0) items.
- In the SLR(1) parsing, we place the reduce move only in the follow of left hand side.

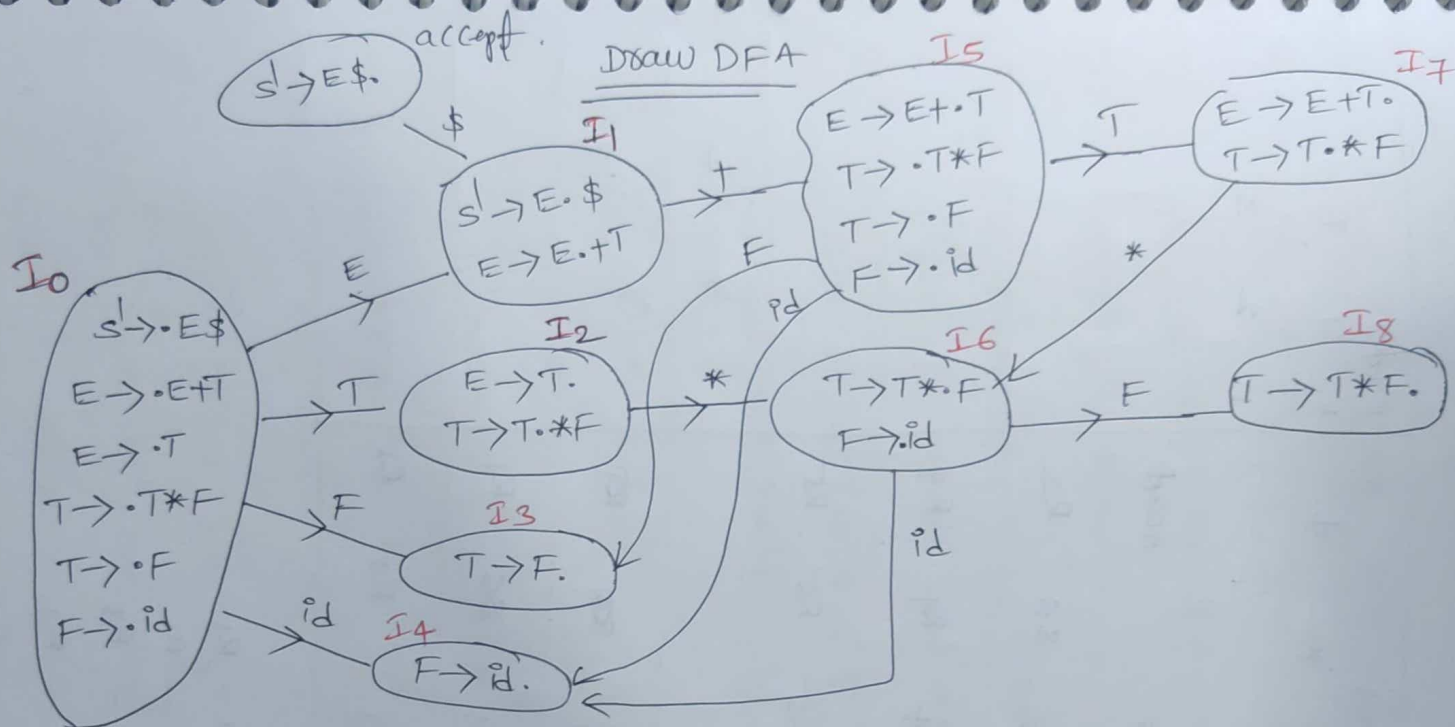
Various steps involved in the SLR(1) parsing

- Check the ambiguity of the grammar
- Add Augment production in the given grammar
- Create canonical collection of LR(0) items.
- Draw data flow diagram (DFA)
- Construct a SLR(1) parsing Table.

Ex 1:- The given CFG is ~~$E \rightarrow E * E$~~ , $E \rightarrow E + T / T$
 $T \rightarrow T * F / F$, $F \rightarrow id$.

Sol:- Add Augment production and insert "." symbol at first position for every production in G.

Draw DFA



SLR(1) Table

States	Actions				Goto		
	id	+	*	\$	E	T	F
I ₀	S ₄				1	2	3
I ₁		S ₅		Accept			
I ₂		R ₂	S ₆	R ₂			
I ₃		R ₄	R ₄	R ₄			
I ₄	S₄	R ₅	R ₅	R ₅			
I ₅	S ₄					7	3
I ₆	S ₄	R₁	S₆	R₁			8
I ₇		R ₁	S ₆	R ₁			
I ₈		R ₃	R ₃	R ₃			

$\text{id} \rightarrow E$
 $\rightarrow E \rightarrow E+T - R_1$
 $E \rightarrow T - R_2$
 $T \rightarrow T * F - R_3$
 $T \rightarrow F - R_4$
 $F \rightarrow \text{id} - R_5$

→ In DFA I_2 has final item that is $E \rightarrow T$ so we have to calculate

$\text{follow}(E) = \{+, \$\}$, so in I_2 now you can put reduction in '+' and '\$' only that is R_2

→ In DFA I_3 has final item that is $T \rightarrow F$. So we have to calculate

$\text{follow}(T) = \{+, \$, *\}$, R_4 - Reduction you can put in '+', '\$', '*' only.

→ In DFA I_4 has final item that is $F \rightarrow \text{id}$. calculate $\text{follow}(F)$

$$\text{follow}(F) = \{+, \$, *, \}$$

→ In DFA I_7 has final item that is $E \rightarrow E+T$. calculate $\text{Follow}(E)$

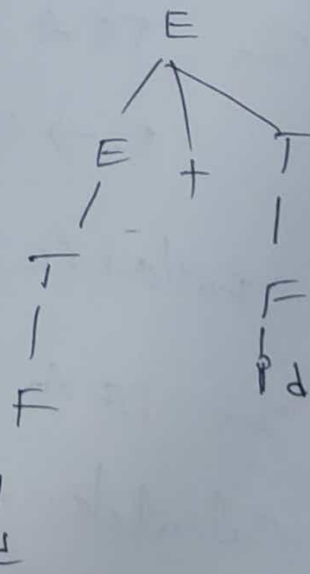
$$\text{Follow}(E) = \{+, \$\}$$

→ In DFA I_8 has final item $T \rightarrow T * F$ calculate $\text{follow}(T) = \{+, \$, *\}$

SLR Parsing

$$w = id + id \$$$

Stack	input	Action
0	id + id \$	shift
oid \$	+ id \$	reduce by $F \rightarrow id$ (1x2x3)
OF \$	+ id \$	reduce by $T \rightarrow F$
OT \$	+ id \$	reduce by $E \rightarrow T$
OEI	+ id \$	shift
OEI + S	id \$	shift
OEI + S id \$	\$	Reduce by $F \rightarrow id$
OEI + S F \$	\$	Reduce by $T \rightarrow F$
OEI + S T \$	\$	Reduce by $E \rightarrow E + T$
OEI acc	\$	accept



CLRC(1) Parsing

→ CLR refers to Canonical lookahead. CLR parsing use the canonical collection of LR(1) items to build the CLRC(1) parsing table.

→ CLRC(1) parsing table produces the more number of states as compare to the SLRC(1) parsing.

→ In the CLRC(1), we place the reduce mode only in the lookahead symbol.

Various steps involved in the CLRC(1) parsing

→ check the ambiguity of the grammar

→ Add Augment productions in the given grammar

→ Create LR(1) items that is

$$LR(1) \text{ items} = LR(0) \text{ items} + \text{look head}$$

→ Draw data flow diagram (DFD)

→ Construct CLRC(1) parsing table.

Example - CLRC(1) Grammar

$$S \rightarrow AA$$

$$A \rightarrow aA$$

$$A \rightarrow b$$

→ Add Augment production insert '.' symbol at the first position for every production in G and also add the look head.

$$I_0 = \begin{array}{l} S' \rightarrow \cdot S, \$ \\ S \rightarrow \cdot AA, \$ \\ A \rightarrow \cdot aA, a|b \\ A \rightarrow \cdot b, a|b \end{array} \quad \begin{array}{l} \text{First}(\$) = \{\$ \} \\ \text{First}(A) = \{a, b\} \end{array}$$

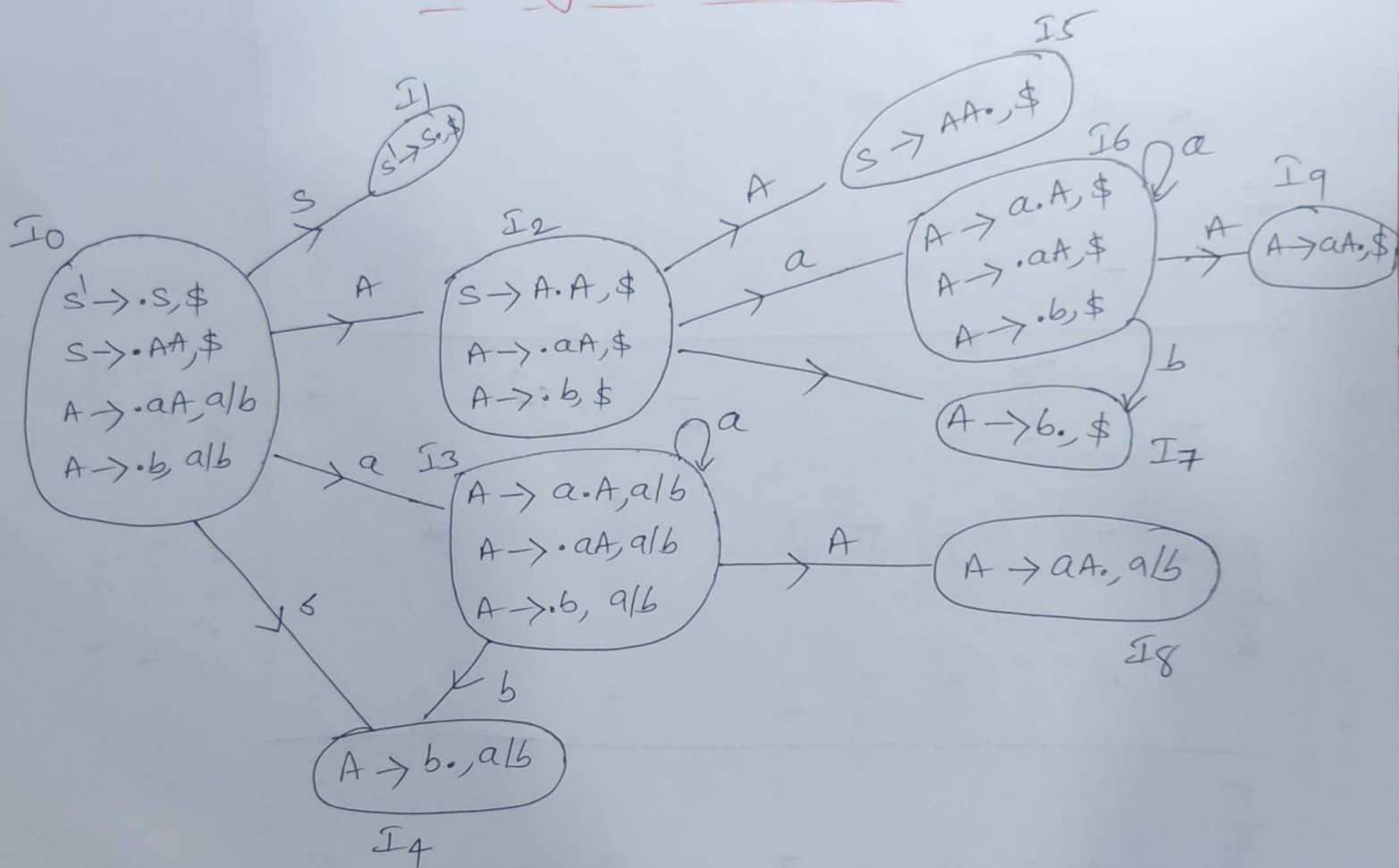
$$I_1 = \text{Goto}(I_0, S) = \text{closure}(S' \rightarrow S \cdot, \$) = S' \rightarrow S \cdot, \$$$

$$I_2 = \text{Goto}(I_0, A) = \text{closure}(S \rightarrow A \cdot A, \$),$$

add all production starting with A in I_2 state
because "." is followed by the non-terminal
so that I_2 state become

$$I_2 = \begin{array}{l} S \rightarrow A \cdot A, \$ \\ A \rightarrow \cdot aA, \$ \\ A \rightarrow \cdot b, \$ \end{array}$$

Drawing DFA for LR(1) Parsing



CLR(1) parsing Table

states	Actions			Goto	
	a	b	\$	S	A
I ₀	S ₃	S ₄		1	2
I ₁			accept		
I ₂	S ₆	S ₇			5
I ₃	S ₃	S ₄			8
I ₄	R ₃	R ₃			
I ₅			R ₁		
I ₆	S ₆	S ₇			9
I ₇			R ₃		
I ₈	R ₂	R ₂			
I ₉			R ₁		

$S \rightarrow AA - R_1$

$A \rightarrow aA - R_2$

$A \rightarrow b - R_3$

CLR(IX) parsing

$w = bb\$$

Stack	input	Action
0	bb\$	shift
0 b 4	b\$	Reduce by $A \rightarrow b$
0 A 2	b\$	shift
0 A 2 b 4	\$	Reduce $A \rightarrow b$
0 A 2 4 5	\$	Reduce $S \rightarrow AA$
0 S 1	\$	
accept		

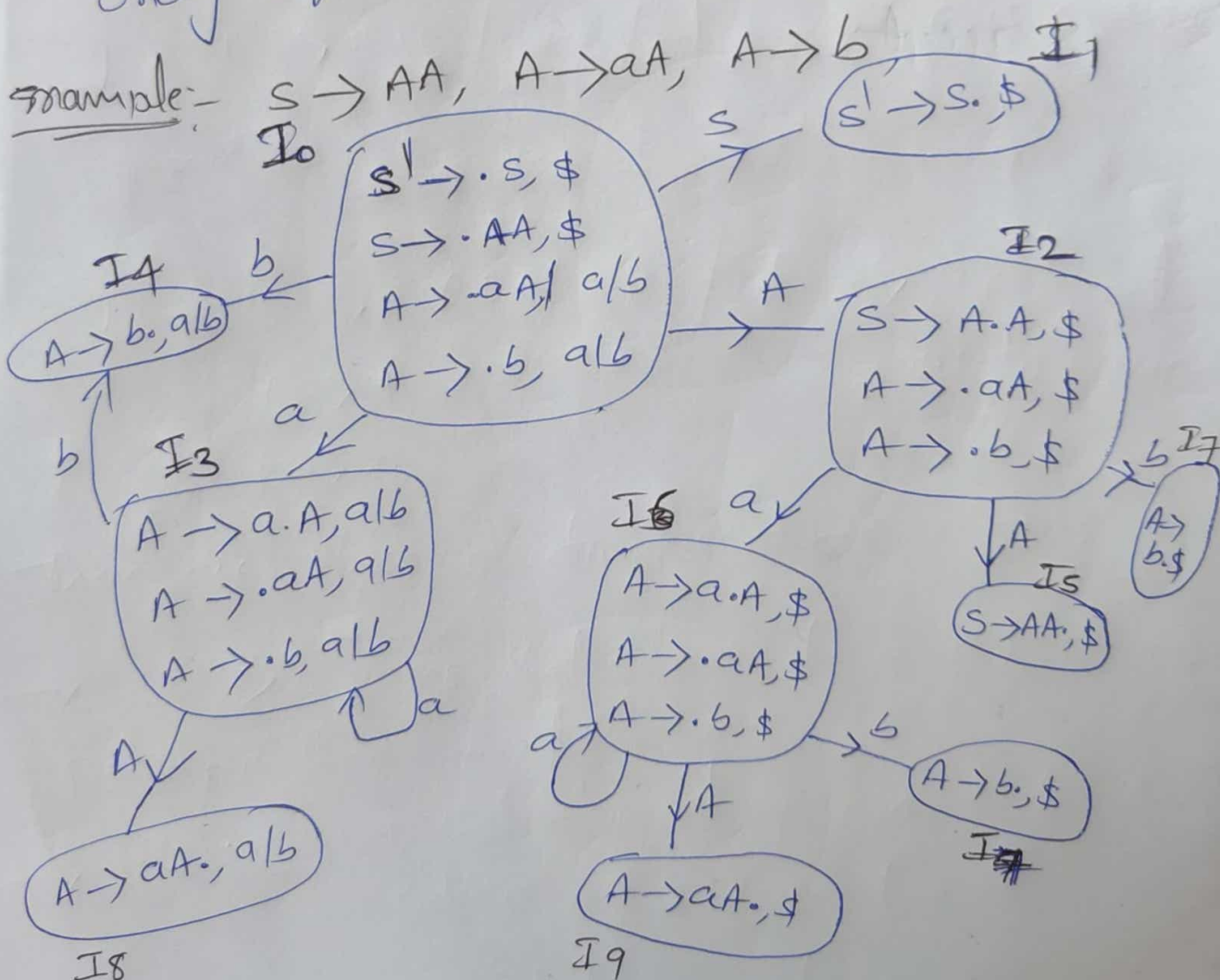
LALR(1) parser

→ LALR refers to the lookahead LR.

→ To construct the LALR(1) parsing table, we use the canonical collection of LR(1) items.

→ In the LALR(1) parsing the LR(1) items which have same productions but different look head are combined to form a single set of items.

→ LALR(1) parsing is same as the LR(1) parsing only difference in the parsing table.



→ The above diagram ~~and~~ table has 9 states
if you combined them we get "6" states.

	a	b	\$	S	A
0	S ₃₆	S ₄₇			2
1				1	
2	S ₃₆	S ₄₇			5
36	S ₃₆	S ₄₇			89
47	r ₃	r ₃	r ₃		
5			r ₁		
89	r ₂	r ₂	r ₂		

* operator precedence parsing

→ operator precedence grammar is kind of shift reduce parsing method. It is applied to a small class of operator grammars.

→ A grammar is said to be operator precedence grammar if it has two properties.

- (1) No R.H.S of any production has ϵ
- (2) No two non-terminals are adjacent

eg:- $E \rightarrow E + E / E * E / id \rightarrow$ It is operator grammar.

eg2:- $E \rightarrow E A E / E B E / id \rightarrow$ It is not operator grammar.
 $A \rightarrow +$
 $B \rightarrow *$

There are three operator precedence relations

→ $a \succ b$ means that terminal 'a' has the higher precedence than terminal 'b'

→ $a \prec b$ means that terminal 'a' has the lower precedence than terminal 'b'

→ $a \doteq b$ means that terminal 'a' and 'b' both are same precedence

eg 1: $E \rightarrow E + E / E * E / id$

→ First we have to construct the operators precedence table.

	id	+	*	\$
id	-	>	>	>
+	<	>	<	>
*	<	>	>	>
\$	<	<	<	-

Stack

input

Action

\$ id

id + id * id \$
↑ ↑

push - id

\$ < . id, push id

id + id * id \$
↑

id > +

pop - id mem
Reduction

$E \rightarrow id$

\$ id

id + id * id
↑

\$ < . +

push '+'

\$ +

+ < . id

push "id"

\$ + id

id + id * id \$
↑

id > * &

pop id & reduce
 $E \rightarrow E * E$

\$ + id

\$	+	*
----	---	---

id + id * id \$
 ↑
 + < . *

push "

\$	+	*	id
----	---	---	----

* < . id push id

id + id * id \$
 ↑

\$	+	*	id
----	---	---	----

id > \$

pop "id" reduce
 $E \rightarrow id$

\$	+	*
----	---	---

* > \$

pop "*" reduction
 $E \rightarrow E * E$

\$	+
----	---

+ > \$

pop "+" reduce
 $E \rightarrow E + E$

\$

\$

If is accepted.

